AETHERGIT: SECTION 3 — FORMALIZATION

Formal Mathematical Modeling of State Transitions and Derived Data Calculus in Agent-Scale Monorepos

To rigorously model source control architectures operating under high-throughput algorithmic modifications in the **AetherGit** framework, we formulate a graph-theoretic foundation. We define the repository state evolution not as an absolute sequence of global tree snapshots, but as an acyclic state-transition system driven by lock-free changesets and recursively evaluated derived data functions.

3.1 GRAPH TOPOLOGY AND CHANGESET ABSTRACTION IN AETHERGIT

DEFINITION 1 (REPOSITORY STATE SPACE)

Let the repository history be represented as a Directed Acyclic Graph (DAG) $G = (V, E)$, where the set of vertices $V = \{c_0, c_1, c_2, ...\}$ corresponds to the set of immutable changesets, and the set of directed edges $E \subset V \times V$ defines explicit parent-child derivation dependencies. A directed edge $(c_i, c_j) \in E$ indicates that c_i is a direct parent ancestor of c_j.

Unlike traditional Version Control Systems (VCS) where a node c encapsulates a full pointer structure to an entire file tree T_c, a changeset c_i in our high-throughput model is strictly defined as a delta-tuple over content-addressed storage blobs:

$$c_i = \langle P(c_i), \Delta_i, t_i, sig_i \rangle \tag{1}$$

Where $P(c_i) = \{p_1, p_2, ..., p_k\} \subset V$ represents the ordered set of parent changesets, t_i is the monotonic logical timestamp, sig_i is the agent cryptographically signed identity, and Δ_i is the localized modification delta set defined over the path namespace Ω:

$$\Delta_i = \{ (path, hash(B)) \mid path \in \Omega, B \in Blobs \} \tag{2}$$

3.2 STATE PROPAGATION AND DERIVED DATA CALCULUS

To eliminate cross-server locking mechanisms during state mutations, we decouple the primary write-ahead log (WAL) ingestion from administrative query structures (such as path histories and blame graphs). Any metadata attribute D required by autonomous agents is formalized as a **Derived Data Function** f_D evaluated recursively over the graph G.

DEFINITION 2 (RECURSIVE DERIVED STATE)

For any derived data domain script{D}, the state $D(c_i)$ associated with changeset c_i is uniquely specified by the recursive composition function $\oplus_D$ over its immediate parent set $P(c_i)$ combined with its local delta Δ_i:

$$D(c_i) = \oplus_D (\{ D(p) \mid p \in P(c_i) \}, \Delta_i) \tag{3}$$

Because $\oplus_D$ is associative and deterministic, evaluation of derived metrics (e.g., path lineage, line-level attribution) satisfies monotonic convergence:

$$\forall\, c_i, c_j \in V, (c_i \preceq c_j) \Rightarrow D(c_i) \subseteq_{sem} D(c_j) \tag{4}$$

3.3 CACHE HIT-RATE PROBABILITY BOUNDS IN ALWAYS-ONLINE TOPOLOGIES

Given an always-online client topology featuring a tiered hierarchy of regional edge caches (C_1), node daemons (C_2), and authoritative storage (S), we model the request service latency L for sparse checkout operations.

Let script{R} be a sparse tree query requested by an autonomous agent execution sandbox targeting a path subset $\Omega_{agent} \subset \Omega$. The probability $P(\text{Hit})$ of resolving tree objects without querying the authoritative Spanner storage tier S is bounded by:

$$P(\text{Hit}) = 1 - \prod_{k=1}^{N} (1 - h_k) \geq 1 - \varepsilon \tag{5}$$

Where h_k denotes the local cache hit ratio at tier k, and ε represents the upper bound on cache miss probability under stationary access distributions. Because content addressing enforces strict immutability (i.e., $hash(Object) = Constant$), cache invalidation overhead is strictly zero:

$$\partial\, CacheState / \partial\, Time = 0 \text{ (for all materialized objects)} \tag{6}$$

Symbol	Mathematical Domain	Physical / Structural Interpretation
$G = (V, E)$	Directed Acyclic Graph	Complete immutable commit ancestry graph
Δ_i	Finite Map: $\Omega \rightarrow$ Hash	Sparse delta mutation set per changeset
$\oplus_D$	Binary Operator: script{D} $\times \Delta \rightarrow$ script{D}	Recursive derivation operator for metadata
Ω_{agent}	Subset of Namespace Ω	Virtual/Sparse checkout path boundary